

Plot in a Pot

Um Editor Visual e Simulador para Histórias
Interativas e Lógica Narrativa

Miguel Gomes Silva

Student No. 2230455

Tomás Alexandre dos Santos Reis Lopes

Supervisor: Anabela Gonçalves Rodrigues Marto

*Professora Adjunta, Escola Superior de
Tecnologia e Gestão do Politécnico de Leiria*

Co-supervisor: Miguel Cerdeira Marreiros Negrão

*Professor Adjunto, Escola Superior de
Tecnologia e Gestão do Politécnico de Leiria*

Nelson Martins Ferreira

*Professor Adjunto, Escola Superior de
Tecnologia e Gestão do Politécnico de Leiria*

Escola Superior de Tecnologia e Gestão
Departamento de Engenharia Informática
Licenciatura em Engenharia Informática

Relatório de Projeto

Leiria, Junho 2026

Plot in a Pot

Copyright © 2026 - Miguel Gomes Silva, Escola Superior de Tecnologia e Gestão.

A presente dissertação é um trabalho original, elaborado exclusivamente para este fim, tendo sido devidamente citados todos os autores cujos estudos contribuíram para a sua elaboração. É permitida a sua reprodução parcial com indicação do autor e referência ao grau, ano letivo, instituição—*Politécnico de Leiria*—e data da defesa pública.

O presente trabalho beneficiou da utilização do modelo *IPLeiria-Thesis*.

Agradecimentos

i Orientação Para a Escrita

In the *Acknowledgment* section, express your gratitude to those who helped and supported your work. Start by thanking your advisors, mentors, or supervisors who provided guidance and expertise. Mention any colleagues, classmates, or team members who contributed to discussions or offered assistance. You can also acknowledge specific organisations, institutions, or funding sources that supported your research or work. Lastly, include any personal acknowledgments for family or friends who offered encouragement and moral support during the project. Keep this section sincere, concise, and professional.

Resumo

i Orientação Para a Escrita

Na secção *Resumo*, apresente um resumo conciso do seu projeto, destacando os pontos principais. Comece com uma breve declaração do problema ou objetivo, seguido de uma descrição da sua abordagem ou metodologia. Resuma os principais resultados ou conclusões, salientando a sua importância ou implicações. Conclua com uma ou duas frases sobre a contribuição global ou o impacto do seu trabalho. O resumo deve ser claro e conciso, idealmente com 150-250 palavras, para que os leitores compreendam rapidamente o seu trabalho e a sua importância.

Palavras-Chave: Palavra-Chave A, Palavra-Chave B, Palavra-Chave C.

Abstract

i Orientação Para a Escrita

In the *Abstract* section, provide a concise summary of your project, highlighting the key points. Begin with a brief statement of the problem or objective, followed by a description of your approach or methodology. Summarise the main results or findings, emphasising their significance or implications. Conclude with a sentence or two on the overall contribution or impact of your work. Keep the abstract clear and focused, ideally within 150-250 words, to give readers a quick understanding of your research and its importance.

Keywords: Keyword A, Keyword B, Keyword C.

Declaração sobre o Uso de Inteligência Artificial

i Orientação Para a Escrita

É uma boa prática académica indicar brevemente como, porquê e quando a IA foi utilizada. Explicar as ações tomadas e as razões que lhes estão subjacentes promove a reflexão crítica, aprofunda a compreensão do papel da IA na aprendizagem e ajuda a avaliar o seu impacto na qualidade e integridade do trabalho. Esta página pode ser removida definindo a opção `aiack` como `false` nas opções da classe do documento.

i Exemplo de Utilização

Reconheço a utilização do ChatGPT (<https://chatgpt.com>) para aperfeiçoar o tom académico e melhorar a precisão linguística deste trabalho, incluindo aspectos de gramática, pontuação e vocabulário.

Descrição da Utilização

Instrução 1: Num parágrafo, descreva simplesmente o que é o LaTeX.

Resposta 1: O LaTeX é um sistema de composição de alta qualidade, normalmente utilizado para produzir documentos científicos e matemáticos, devido ao seu poderoso manuseamento de fórmulas e bibliografias. Ao contrário dos processadores de texto, utiliza ficheiros de texto simples com etiquetas de marcação para definir a estrutura e a formatação de um documento, permitindo um controlo preciso da disposição e do aspeto. É especialmente popular no meio académico e na investigação para a criação de teses, artigos de revistas e apresentações.

Conteúdo

<i>Lista de Figuras</i>	xii
<i>Lista de Tabelas</i>	xiv
<i>Glossário</i>	xvi
<i>Siglas</i>	xviii
<i>Símbolos</i>	xx
1 Introdução	1
1.1 Objeto do Trabalho	1
1.2 Justificação e Pertinência do Tema	1
1.3 Objetivos do Trabalho	2
1.3.1 Objetivo Geral	2
1.3.2 Objetivos Específicos	2
1.4 Requisitos Gerais do Sistema	3
1.5 Métodos e Técnicas Utilizadas	3
1.6 Estrutura do Documento	4
2 Fundamentação Teórica e Estado da Arte	5
2.1 Teoria de Grafos na Narrativa Interativa	5
2.1.1 Tipos de Grafos e Aplicação Narrativa	5
2.1.2 Grafos Direcionados Acíclicos (DAGs) vs. Grafos com Ciclos	6
2.2 Estado da Arte	7
2.2.1 Twine	7
2.2.2 Ink (Inkle Studios)	7
2.2.3 ChoiceScript	8
2.2.4 Yarn Spinner	8
2.2.5 Narrative Flow	8
2.2.6 Articy Draft	8
2.2.7 Análise Comparativa	9
2.3 Integração de Modelos de Linguagem de Grande Escala (LLMs) Locais	10
2.3.1 Análise de Famílias de Modelos e Recomendações	10
2.3.2 Modelo de Integração Local sem Custos	11
2.3.3 Papel do LLM: Conversão e Importação Automática de Histórias	12

3	Conceção e Implementação	13
3.1	Requisitos Funcionais e Não Funcionais	13
3.1.1	Requisitos Funcionais	13
3.1.2	Requisitos Não Funcionais	14
3.2	Arquitetura Geral do Sistema	14
3.3	Modelação de Dados: Lista de Adjacência Bidirecional	15
3.4	Parser Síncrono e Especificação Twee 3	16
3.4.1	Importação e Conversão de Coordenadas em Zonas	16
3.4.2	Passagens Especiais	16
3.4.3	Modelo SugarCube (Ligações e Macros)	16
3.5	Interação Visual, Edição e Acessibilidade	17
3.5.1	Zonas de Agrupamento Visual	17
3.5.2	Arestas Curvadas com Waypoints	17
3.5.3	Gestão de Variáveis Estilo Figma	17
3.5.4	Acessibilidade para Autores Daltónicos	18
3.6	Validação Estática e Simulação	18
3.6.1	Validador Estático de Lógica (BFS)	18
3.6.2	Simulador em Tempo Real (PlayMode) e Smart Walker	18
4	Conclusão e Trabalho Futuro	20
4.1	Conclusões	20
4.2	Limitações da Implementação	20
4.3	Trabalho Futuro	21
	<i>Bibliography</i>	23
	Apêndices	
A	Showcasing the First Appendix	28
B	Showcasing the Second Appendix	29
	Anexos	
L	Showcasing the First Annex	33

Lista de Figuras

Lista de Tabelas

2.1	Análise comparativa de motores de ficção interativa.	9
-----	--	---

1

Introdução

O desenvolvimento de jogos e narrativas interativas tem vindo a ganhar uma importância crescente no panorama do entretenimento e da educação. Diferente do cinema ou da literatura tradicional, a Ficção Interativa (FI) coloca o utilizador no centro da tomada de decisões, permitindo-lhe moldar o rumo dos acontecimentos através das suas escolhas. Esta não-linearidade narrativa, no entanto, introduz uma complexidade acrescida na escrita e no planeamento das histórias, exigindo ferramentas capazes de gerir fluxos de informação complexos e lógica de estados dinâmicos.

Neste contexto surge o **Plot in a Pot**, um editor visual e simulador de narrativas interativas. O projeto foi concebido para resolver os principais desafios enfrentados pelos autores de histórias não-lineares, combinando uma interface visual de alto desempenho baseada em grafos direcionados com um motor de simulação que suporta a execução de código e lógica em tempo real.

1.1 Objeto do Trabalho

O objeto deste projeto consiste no desenvolvimento do **Plot in a Pot (PiaP)**, uma aplicação e ambiente de desenvolvimento concebido para a criação, estruturação, simulação e tradução de narrativas interativas e não-lineares. O sistema foca-se na integração e suporte nativo ao ecossistema do motor Twine, operando especificamente com o formato de ficheiros Tweep (versão 3) e com a macro-linguagem SugarCube. Através de uma interface gráfica baseada em grafos, a ferramenta permite a autores e *game designers* gerir ramificações complexas e bases de dados de localização de uma forma unificada.

1.2 Justificação e Pertinência do Tema

O *design* de narrativas para jogos não-lineares enfrenta frequentemente problemas severos de escala, onde árvores de diálogo e escolhas crescem exponencialmente, tornando-se difíceis de rastrear e propensas a erros lógicos. Embora o Twine seja uma das ferramentas de referência da comunidade independente e académica (**Interactive Fiction Technology**

Foundation, 2026) para a prototipagem rápida de ficção interativa, o desenvolvimento de projetos profissionais de grande escala é severamente limitado pela falta de ferramentas nativas para a gestão multi-idioma (localização) e pela ausência de validadores estáticos de fluxo que previnam caminhos mortos antes da compilação.

O PiaP justifica-se ao resolver esta lacuna, introduzindo uma estética neo-brutalista de alto contraste que otimiza o espaço de trabalho, permitindo isolar cadeias de texto rígidas (*hardcoded*) através de um sistema abstrato de chaves de tradução dinâmicas e oferecendo segurança matemática no mapeamento de rotas narrativas.

1.3 Objetivos do Trabalho

1.3.1 Objetivo Geral

Desenvolver uma plataforma *web* modular que unifique a edição visual de grafos de histórias interativas Twee com um motor (*engine*) integrado de localização multi-idioma e validação de caminhos lógicos.

1.3.2 Objetivos Específicos

Para a concretização do objetivo geral, definiram-se os seguintes objetivos específicos:

1. **Interface Visual Neo-Brutalista e Acessível:** Implementar um *canvas* de grafos interativo e de alto desempenho no estilo neo-brutalista com forte contraste visual, que suporte a criação de nós de cena, zonas de agrupamento visual, *scripts* (JavaScript) e folhas de estilo (CSS), facilitando a utilização por pessoas com daltonismo.
2. **Otimização Visual com Waypoints:** Desenvolver um algoritmo de rota iterativo através de *waypoints* geométricos nas arestas para otimização visual e organização do espaço de trabalho.
3. **Parser Bidirecional Twee3:** Construir um analisador (*parser*) bidirecional síncrono em tempo real capaz de ler a sintaxe Twine (`[[LinkDestino]]`) e reconstruir a topologia do grafo instantaneamente, mantendo a compatibilidade e convertendo coordenadas.
4. **Matriz de Localização Integrada:** Criar uma matriz de localização baseada em grelha *Flexbox* com suporte para importação/exportação em formato CSV e tradução *inline* por células.
5. **Validador de Fluxo baseado em Listas de Adjacência:** Desenvolver um motor de verificação estática baseado em listas de adjacência para detetar nós inacessíveis/órfãos, becos sem saída ou *loops* infinitos na narrativa.
6. **Simulador Imersivo e Tutorial Jogável:** Implementar um ambiente de simulação em tempo real (*PlayMode*) e um tutorial jogável (*Playable Tutorial*) para validação pedagógica das regras do editor.

1.4 Requisitos Gerais do Sistema

Com o intuito de delimitar o âmbito do projeto e as necessidades fundamentais a que o **PiaP** responde, foram estabelecidos os seguintes requisitos gerais que guiam o comportamento e a utilidade da aplicação:

- **Modelador Narrativo Visual:** A aplicação deve disponibilizar uma interface gráfica baseada em grafos que permita modelar e ligar de forma dinâmica as cenas da história.
- **Motor de Tradução Integrado:** Deve fornecer um sistema para traduzir e localizar textos diretamente na ferramenta de autoria, eliminando a dispersão de ficheiros externos.
- **Análise e Validação de Consistência:** O sistema deve auditar as ligações criadas para detetar inconsistências como becos sem saída, nós isolados ou ciclos sem saída apropriada.
- **Compatibilidade com o Ecossistema Twine:** O software deve garantir interoperabilidade bidirecional com o formato padrão Tweak 3 (SugarCube), permitindo importar e exportar ficheiros sem perda de metadados.
- **Simulação em Tempo Real:** Fornecer um modo interativo para jogar, depurar e testar a narrativa diretamente na ferramenta sem necessidade de compilação externa.
- **Conversão Inteligente de Texto via IA:** O sistema deve permitir que autores escrevam histórias corridas em linguagem natural e convertam esses textos em grafos estruturados no formato Tweak automaticamente usando inteligência artificial local.

1.5 Métodos e Técnicas Utilizadas

O desenvolvimento do projeto apoiou-se em metodologias ágeis com ciclos rápidos de prototipagem e testes de esforço (*stress*) de dados. Ao nível tecnológico e arquitetural, foram utilizadas as seguintes ferramentas e conceitos:

- **Frontend e Canvas de Grafos:** Desenvolvimento baseado na biblioteca React (Meta Open Source, 2013), devido a ser uma das mais populares (Statista Research Department, 2025), utilizando o ecossistema React Flow (webkid.io, 2021) para a renderização, manipulação e cálculo de coordenadas de grafos, escolhido pela sua integração nativa com React.
- **Algoritmia e Teoria de Grafos:** Utilização de estruturas de dados baseadas em listas de adjacências para a implementação de algoritmos de pesquisa (BFS e DFS) que validam de forma estática e iterativa a integridade dos caminhos e simulam estados de variáveis em memória.
- **Internacionalização e Parsing:** Integração da biblioteca i18next em conjunto com algoritmos de substituição baseados em Expressões Regulares (Regexs) para

interceptar macros SugarCube e injetar as chaves da base de dados de localização em tempo real.

1.6 Estrutura do Documento

Este relatório está estruturado em quatro capítulos principais. Além desta introdução, o documento apresenta:

- **Capítulo 2: Fundamentação Teórica e Estado da Arte**, onde se efetua uma análise comparativa das ferramentas existentes no mercado e se descrevem os conceitos matemáticos de teoria de grafos aplicados à narrativa.
- **Capítulo 3: Conceção e Implementação do Plot in a Pot**, detalhando a arquitetura do software, a especificação Twee3, o modelo de lista de adjacências bidirecional, os modos de visualização e acessibilidade, e a simulação lógica.
- **Capítulo 4: Conclusão e Trabalho Futuro**, onde são sumariados os resultados alcançados, as limitações da implementação e as propostas para futuros desenvolvimentos.

2

Fundamentação Teórica e Estado da Arte

Neste capítulo são expostos os fundamentos teóricos relativos à modelação matemática de fluxos narrativos interativos, bem como a análise do estado da arte sobre as ferramentas de autoria existentes, destacando os seus pontos fortes e limitações.

2.1 Teoria de Grafos na Narrativa Interativa

A estrutura básica de uma história interativa assenta na modelação geométrica e lógica de caminhos narrativos. Um fluxo de história pode ser formalmente representado como um Grafo $G = (V, E)$ (Bondy et al., 2008), onde:

- V representa o conjunto de **Vértices** (ou nós), que correspondem a cenas, passagens de texto ou estados lógicos da história.
- E representa o conjunto de **Arestas** (ou ligações direcionadas), que representam as escolhas disponibilizadas ao leitor para transitar de uma cena para a outra.

2.1.1 Tipos de Grafos e Aplicação Narrativa

Dependendo da natureza do jogo ou da história, utilizam-se diferentes tipos de estruturas de grafos:

1. **Grafo Direcionado (Directed Graph):** Uma estrutura onde as ligações têm direção, ou seja, a existência de um caminho de $A \rightarrow B$ não implica o retorno de $B \rightarrow A$. Na matriz de adjacência, isto traduz-se numa estrutura não-simétrica:

$$M[i][j] \neq M[j][i]$$

Representa o fluxo narrativo puro, onde cada escolha conduz o leitor numa direção específica.

2. **Grafo Não-Direcionado (Undirected Graph):** Representa relações simétricas e bidirecionais ($A - B$). Embora seja menos comum em narrativa linear, é útil para modelar redes de relacionamento social entre personagens ou conexões espaciais de livre exploração (ex: salas adjacentes de uma masmorra). Na matriz de adjacência, a estrutura é simétrica:

$$M[i][j] = M[j][i]$$

3. **Grafo de Conhecimento (Knowledge Graph):** Modela relações semânticas complexas entre entidades (ex: “personagem conhece objeto”, “porta depende de chave”). Exige múltiplas matrizes de adjacência ou uma estrutura tridimensional:

$$M_{relation}[i][j]$$

É ideal para jogos com inventários complexos e decisões baseadas em conhecimento indireto acumulado pelo utilizador.

4. **Grafo Ponderado (Weighted Graph):** Atribui valores (pesos) às arestas:

$$M[i][j] = \text{peso}$$

Estes pesos podem representar custos de recursos (tempo, pontos de vida), custos emocionais do personagem, ou a probabilidade de uma escolha ter sucesso.

2.1.2 Grafos Direcionados Acíclicos (DAGs) vs. Grafos com Ciclos

Tradicionalmente, muitas plataformas limitam a escrita a Grafos Direcionados Acíclicos (*Directed Acyclic Graphs* - DAGs). A propriedade acíclica garante que é impossível iniciar num vértice V_i e, através de arestas direcionadas, regressar a V_i . Isto garante um fluxo estritamente unidirecional e progressivo de causa e efeito, facilitando a aplicação de algoritmos como a Ordenação Topológica (*Topological Sorting*) com complexidade linear:

$$O(V + E)$$

A ordenação topológica (através de abordagens como o algoritmo de Kahn (Kahn, 1962)) alinha os vértices linearmente, facilitando o cálculo imediato do progresso da história e a identificação do caminho mais rápido.

No entanto, em narrativas complexas, a imposição de DAGs é altamente limitadora. A impossibilidade de introduzir ciclos impede a criação de:

- **Estruturas de Cubo (Hubs):** Onde o jogador explora vários caminhos secundários a partir de uma praça ou menu central e regressa recorrentemente ao mesmo ponto.
- **Mecânicas de Simulação e Tempo:** Eventos baseados em estados lógicos recorrentes, como “aguardar um turno” no mesmo local para fazer avançar variáveis temporais.

Por esta razão, o **Plot in a Pot** adota um modelo de **Grafo Direcionado Geral** que permite ciclos lógicos. Isto requer algoritmos de validação dinâmica em segundo plano (como pesquisas BFS e DFS (Cormen et al., 2009)) para evitar a ocorrência de loops infinitos sem condições de saída logicamente definidas.

2.2 Estado da Arte

O conceito de ficção interativa refere-se a narrativas estruturadas onde a progressão do texto não é linear, dependendo diretamente das escolhas e ações submetidas pelo utilizador para navegar caminhos alternativos (Montfort, 2005). Com a expansão do desenvolvimento de videojogos independentes e o crescente estudo de narratologia computacional, diversas ferramentas de autoria emergiram para suportar a criação deste tipo de estruturas ramificadas, destacando-se o Twine, o Ink e o ChoiceScript (Interactive Fiction Technology Foundation, 2026).

Apresenta-se, de seguida, uma análise detalhada das principais referências do mercado académico e profissional.

2.2.1 Twine

Lançado em 2009 por Chris Klimas (Klimas, 2009), o Twine é a ferramenta mais popular na comunidade de ficção interativa baseada em texto.

- **Vantagens:** Extremamente acessível (sem barreira de programação para escrita básica); visualização clara baseada em nós gráficos no canvas; open-source e gratuito (distribuído sob a licença GPL v3, permitindo qualquer utilização, incluindo comercial, sem restrições); rápido para prototipagem rápida.
- **Limitações:** A interface visual torna-se desorganizada e lenta em projetos muito grandes (efeito “prato de esparguete”); a lógica de estado nativa é limitada; a integração com motores de jogos externos (como Unity ou Unreal) é complexa.

O fluxo de trabalho central do Twine assenta na criação de passagens (segmentos individuais da história) e na sua ligação através de hiperligações que representam escolhas do jogador. Estas passagens são exibidas como caixas num canvas com setas a indicar os caminhos narrativos (TheToolpicker Team, 2026). Este paradigma mapeia-se diretamente para a representação em canvas de grafos interativos que constitui o núcleo visual do **PiaP**, tornando a correspondência entre o modelo de dados interno e a interface imediata e semanticamente coerente.

2.2.2 Ink (Inkle Studios)

O Ink (Inkle Studios, 2016) é uma linguagem textual de scripting criada pela Inkle Studios (criadores de jogos como *80 Days*).

- **Vantagens:** Altamente poderosa para controlo fino de estados narrativos, condicionais complexas e ramificações complexas; excelente integração técnica com

motores de jogo; licenciada sob a licença MIT (totalmente livre); escala muito bem para projetos de grande envergadura.

- **Limitações:** Não é uma ferramenta visual (opera de forma textual como scripting acoplado a motores de jogo), o que dificulta o planeamento geográfico e a deteção de redundâncias de caminhos por autores não-técnicos; exige conhecimentos de lógica de programação.

2.2.3 ChoiceScript

Desenvolvido pela Choice of Games, o ChoiceScript é uma linguagem de programação simples e baseada em texto para a escrita de romances interativos baseados em escolhas.

- **Vantagens:** Sintaxe simples e linear de fácil compreensão para escritores; possui excelentes ferramentas de teste automático (como `quicktest` e `randomtest`) que facilitam a validação de caminhos e a robustez lógica de romances longos.
- **Limitações:** Não é uma ferramenta visual; o formato de ficheiro é proprietário; a sua licença (CSL) proíbe explicitamente a utilização comercial sem autorização prévia e acordo de distribuição comercial com a Choice of Games ([Choice of Games, 2026](#)), limitando a autonomia de autores independentes.

2.2.4 Yarn Spinner

O Yarn Spinner ([Secret Lab, 2015](#)) é uma ferramenta focada na integração de diálogos em jogos interativos com motor gráfico.

- **Vantagens:** Sintaxe simples (semelhante a scripts de teatro); suporte nativo para variáveis; excelente portabilidade direta para Unity e Unreal Engine.
- **Limitações:** Carece de um planeador visual abstrato robusto fora do motor de jogo principal; não é adequado para desenhar fluxogramas abstratos de histórias antes da fase de produção do jogo.

2.2.5 Narrative Flow

O Narrative Flow ([NarrativeFlow, 2023](#)) é uma ferramenta emergente que tenta colmatar a lacuna entre Twine e Ink.

- **Vantagens:** Oferece melhor organização estrutural do que o Twine e uma visualização mais clara que o Ink; ajuda a gerir a consistência da escrita.
- **Limitações:** É um software pago, com menor maturidade técnica e uma comunidade de utilizadores consideravelmente mais reduzida.

2.2.6 Articy Draft

O Articy Draft ([Articy Software, 2010](#)) é a solução comercial líder da indústria de videogames AAA para planeamento de narrativas e bases de dados de escrita.

- **Vantagens:** Altamente estruturado e profissional; gestão integrada de personagens, inventários e diálogos; escala com eficácia para equipas de desenvolvimento de grandes dimensões.
- **Limitações:** Curva de aprendizagem acentuada; software dispendioso (licença comercial paga); excessivamente complexo e pesado para autores independentes ou projetos pequenos.

2.2.7 Análise Comparativa

A fim de estruturar a comparação entre as diferentes abordagens, apresenta-se na Tabela 2.1 uma síntese comparativa das três principais ferramentas de ficção interativa baseadas nos critérios extraídos da literatura e da prática de desenvolvimento.

Tabela 2.1: *Análise comparativa de motores de ficção interativa.*

Critério	Twine	Ink/Inky	ChoiceScript
Licença	GPL v3 — livre (incl. comercial)	MIT — totalmente livre	CSL — não-comercial sem licença
Interface de Edição	Visual (grafo de nós/arestas)	Textual (scripting em IDE)	Textual (scripting linear)
Exportação Standalone	HTML autónomo (sem engine)	Requer engine externa (Unity/Unreal)	Web via plataforma CoG
Adoção Académica/Indie	Muito elevada	Média (foco profissional)	Média (foco romances interativos)
Integração com Motores	Limitada (plugins de terceiros)	Nativa (Unity, Unreal — oficial)	Nenhuma
Ferramentas de Teste	Limitadas	Básicas	Avançadas (quicktest, randomtest)
Formato Aberto	Sim (Twee3 — spec pública)	Sim (JSON compilado)	Não (proprietário CoG)
Curva de Aprendizagem	Baixa (visual, sem código)	Alta (linguagem scripting)	Média (sintaxe simples)
Mapeamento para Grafo	Direto (passagem = nó, link = aresta)	Indireto (estrutura textual)	Indireto (estrutura textual)

Esta análise comparativa é de extrema relevância para a conceção do **PiaP**, uma vez que fundamenta diretamente as decisões arquiteturais e os requisitos definidos para o projeto:

- **Fusão de Paradigmas (Visual vs. Textual):** Enquanto o Twine se destaca pela interface puramente visual e o Ink pelo controlo fino de estados lógicos complexos, o **PiaP** visa fundir ambas as forças, proporcionando um editor visual de grafos que não abdica da manipulação robusta de condicionais e variáveis (via macro-linguagem SugarCube).
- **Adoção do Formato Aberto Twee 3 e Limitação de Grafos:** A tabela salienta que o formato Twee 3 é uma especificação aberta com mapeamento direto

entre passagens e grafos, o que suporta a decisão de adotá-lo como base de persistência do **PiaP**. Contudo, para permitir que o sistema leia as saídas e as renderize visualmente como arestas de forma fiável, o projeto introduz um compromisso de design (*trade-off*) em relação ao Twine tradicional: não é suportada a utilização de variáveis como destinos dinâmicos de hiperligações (ex: `[[Texto|$destino]]`), exigindo que todos os destinos do grafo sejam estáticos.

- **Validação Automática e Segurança do Fluxo:** O ChoiceScript é valorizado pelas suas ferramentas robustas de teste automático (`randomtest`), uma lacuna crítica do Twine. O **PiaP** incorpora de raiz este princípio ao disponibilizar o validador estático (BFS) e o Smart Walker no simulador para preencher esta falta, garantindo a correção imediata do fluxo narrativo.
- **Autonomia e Licenciamento Livre:** A restrição de uso comercial imposta pelo ChoiceScript salienta a importância de desenvolver uma ferramenta que promova a autonomia dos autores, permitindo a exportação de projetos sob licenças livres e sem taxas adicionais.

2.3 Integração de Modelos de Linguagem de Grande Escala (LLMs) Locais

Com o avanço recente no campo da Inteligência Artificial Generativa, a integração de Modelos de Linguagem de Grande Escala (*Large Language Models* - LLMs) tornou-se um recurso valioso para auxiliar autores na expansão automática e co-criação de narrativas interativas. No entanto, para garantir a viabilidade prática do projeto **PiaP** sem dependências financeiras ou de conectividade externa, focou-se a pesquisa em modelos de execução local e gratuita.

2.3.1 Análise de Famílias de Modelos e Recomendações

Foram analisadas três das principais famílias de modelos em open-weight ajustados para instruções, avaliando a sua capacidade e adequabilidade para autoria de ficção interativa:

1. Família Llama 3 e 3.1 (Meta AI):

- **Versão Recomendada:** Llama 3.1 (8B).
- **Justificação:** Representa um dos modelos leves mais capazes do estado da arte. A versão de 8 mil milhões de parâmetros corre com excelente desempenho na maioria dos computadores modernos equipados com placa gráfica dedicada ou em arquiteturas Mac Apple Silicon (M1/M2/M3), requerendo investimentos de hardware reduzidos.
- **Vantagem Narrativa:** Demonstra uma capacidade excecional no cumprimento estrito de diretrizes de formatação (*system prompts* rígidos). Esta fiabilidade torna-o ideal para gerar respostas estruturadas (por exemplo, em

formato JSON contendo a sugestão de cena e as escolhas de saída), facilitando a integração direta no motor lógico do software.

2. Família Mistral (Mistral AI):

- **Versão Recomendada:** Mistral NeMo (12B) ou Mistral v0.3 (7B).
- **Justificação:** Os modelos desenvolvidos pela empresa europeia Mistral AI são amplamente reconhecidos pela comunidade de escrita criativa pelo seu tom mais orgânico e natural. Adicionalmente, tendem a apresentar menor nível de censura prévia, facilitando a escrita livre em gêneros literários como ficção geral, mistério ou terror.
- **Vantagem Narrativa:** Apresentam janelas de contexto extremamente eficientes. A janela de contexto atua como a memória de curto prazo do modelo; numa narrativa ramificada, é fundamental que o LLM recorde o histórico de cenas e as decisões anteriores tomadas pelo utilizador para garantir a coerência semântica global do texto gerado.

3. Gemma 2 (Google):

- **Versão Recomendada:** Gemma 2 (9B).
- **Justificação:** Apesar do seu tamanho compacto de 9 mil milhões de parâmetros, destaca-se pelo seu desempenho elevado em raciocínio abstrato e análises textuais comparativas.
- **Vantagem Narrativa:** É altamente eficaz na realização de resumos e sínteses. Esta capacidade é ideal para funcionalidades auxiliares que leiam um determinado ramo completo da história e gerem um resumo conciso (ex: “descreve os principais acontecimentos deste arco narrativo”).

2.3.2 Modelo de Integração Local sem Custos

Para evitar a necessidade de desenvolver e compilar motores de inferência de raiz, o **PiaP** adota uma arquitetura baseada em servidores de inferência locais prontos a usar, nomeadamente o *Ollama* ou o *LM Studio*. O fluxo de comunicação processa-se conforme descrito de seguida:

1. **Instalação Local:** O autor/utilizador instala o servidor *Ollama* na sua máquina local.
2. **Execução em Segundo Plano:** O *Ollama* descarrega o modelo pretendido (ex: llama3.1:8b) e executa a inferência local em segundo plano.
3. **Simulação de API:** O servidor expõe uma API REST local na porta padrão (tipicamente `http://localhost:11434`), que simula a interface padrão de chat da OpenAI.
4. **Pedidos HTTP:** O software *Plot in a Pot* efetua pedidos HTTP padrão (*fetch*) diretamente para este endpoint local, eliminando custos de infraestrutura e limites de chamadas impostos por APIs proprietárias na nuvem.

2.3.3 Papel do LLM: Conversão e Importação Automática de Histórias

Uma conclusão crítica retirada da pesquisa reside no facto de que os LLMs não possuem fiabilidade suficiente para gerir a coerência estrutural de um grafo de estado de forma autónoma e dinâmica em tempo de execução. Os modelos de linguagem tendem a alucinar, esquecer nós, criar ligações logicamente impossíveis, ignorar restrições de ciclos e perder o mapeamento geral da árvore narrativa quando integrados no fluxo direto de jogo.

Deste modo, no produto final, a função do LLM foi redefinida para atuar exclusivamente como uma ferramenta de conversão estrutural numa fase de pré-processamento. Em vez de ser integrado diretamente na simulação, o modelo é utilizado para ler uma história corrida escrita pelo utilizador em linguagem natural e convertê-la inteiramente para o formato normativo Tweep 3 (identificando as passagens/nós, os seus títulos e as arestas de ligação). Uma vez gerada a representação textual Tweep, a aplicação importa-a de forma automática para o programa. A partir desse ponto, o motor lógico proprietário e o validador estático (baseados em listas de adjacências) assumem a responsabilidade exclusiva de desenhar o canvas visual e garantir a coerência matemática do grafo de estados.

3

Conceção e Implementação

Este capítulo descreve a arquitetura geral do **Plot in a Pot**, detalhando as decisões de design de dados, o funcionamento dos algoritmos de processamento, as estratégias de visualização e acessibilidade adotadas, e o motor de simulação implementado.

3.1 Requisitos Funcionais e Não Funcionais

Para a materialização prática do sistema, os objetivos e requisitos gerais delineados na introdução foram detalhados sob a forma de requisitos técnicos. Estes dividem-se em funcionais (o que o sistema deve executar diretamente) e não funcionais (as restrições técnicas, de qualidade e usabilidade sob as quais o sistema deve operar).

3.1.1 Requisitos Funcionais

- RF01 - Criação e Edição de Nós Narrativos (Cenas):** O utilizador deve poder criar, mover, editar (alterando título e conteúdo de texto) e apagar nós no canvas interativo.
- RF02 - Ligações Direcionadas entre Nós (Escolhas):** O sistema deve ler ligações inseridas no texto através da sintaxe double-bracket (`[[TextoDestino]]`) e criar arestas direcionadas automaticamente no grafo.
- RF03 - Zonas de Agrupamento Visual:** O utilizador deve ser capaz de criar áreas retangulares redimensionáveis no canvas (Zonas) para agrupar nós de cena sob um mesmo contexto visual.
- RF04 - Parser e Compatibilidade Twee 3:** O software deve importar e exportar ficheiros no formato textual normativo `.twee` síncrona e bidirecionalmente, preservando a sintaxe Twine e convertendo coordenadas.
- RF05 - Matriz de Localização Multi-idioma:** Disponibilizar uma tabela em formato de grelha (*grid*) que permita traduzir o texto das cenas para diferentes idiomas a partir de chaves dinâmicas, com suporte para exportação e importação em formato CSV.

RF06 - Auditoria e Validação Estática de Fluxo: O sistema deve executar análises estáticas sobre a estrutura do grafo em segundo plano para detetar e alertar sobre nós órfãos (inacessíveis), ligações quebradas (que apontam para nós apagados) ou ciclos infinitos sem condição de saída lógica.

RF07 - Simulador em Tempo Real (PlayMode): O utilizador deve poder testar a história jogando-a numa sandbox isolada dentro da aplicação que interprete as macros lógicas do SugarCube e execute código Javascript e CSS customizado.

RF08 - Importação Inteligente via Conversor LLM: A aplicação deve permitir que o utilizador submeta uma história corrida em linguagem natural e, através de um modelo de linguagem local (Ollama) ou remoto (Gemini), a converta automaticamente em ficheiro no formato Twee 3 para gerar o respetivo grafo.

3.1.2 Requisitos Não Funcionais

RNF01 - Portabilidade e Execução no Cliente (Browser): A plataforma deve ser executável inteiramente no lado do cliente através de um navegador de internet padrão, sem requerer servidores ou infraestruturas complexas.

RNF02 - Usabilidade e Acessibilidade Visual: A interface de grafos deve utilizar a estética neo-brutalista e garantir que a visualização de fluxos (arestas de entrada e saída) seja distinguível sem dependência exclusiva de cor, apoiando utilizadores daltónicos.

RNF03 - Desempenho e Fluidez: O canvas do React Flow e a conversão do parser síncrono devem suportar dezenas de nós em simultâneo com latência de interação e atualização abaixo de 100 milissegundos.

RNF04 - Privacidade e Soberania de Dados: Todos os dados introduzidos, incluindo ficheiros importados, tabelas de tradução e dados submetidos para inteligência artificial no Ollama local, devem permanecer estritamente no armazenamento local do utilizador para garantir segurança absoluta.

3.2 Arquitetura Geral do Sistema

O **Plot in a Pot** foi concebido como uma aplicação web de página única (SPA - *Single Page Application*) desenvolvida em **React 19** (Meta Open Source, 2013). A interface utiliza a biblioteca **ReactFlow 11** (webkid.io, 2021) para o motor gráfico do canvas de grafos, e **Tailwind CSS 3** (Tailwind Labs, 2017) para a aplicação de uma identidade visual neo-brutalista (caracterizada por sombras sólidas pretas, contornos escuros de espessura elevada e ausência de cantos arredondados).

O sistema de internacionalização baseia-se na biblioteca **i18next**, com uma arquitetura de carregamento estático assíncrono. Os dicionários de tradução residem em ficheiros JSON estáticos na diretoria pública da aplicação (`locales/pt/translation.json` e `locales/en/translation.json`), sendo carregados dinamicamente via requisições HTTP em runtime através do módulo **i18next-http-backend**, o que reduz o tama-

nho do bundle principal de JavaScript.

3.3 Modelação de Dados: Lista de Adjacência Bidirecional

Para representar o fluxo da história no editor, evitou-se o uso de uma Matriz de Adjacência convencional devido ao desperdício de memória ($O(V^2)$) e à fraca escalabilidade com o crescimento do número de nós. Em vez disso, implementou-se uma **Lista de Adjacência Bidirecional** com complexidade $O(V + E)$.

Cada cena no modelo de dados armazena de forma independente dois conjuntos de caminhos:

1. **Saídas (Sucessores):** Um vetor que contém as escolhas explícitas disponíveis para o leitor, mapeando o texto da opção e o ID do nó de destino.
2. **Entradas (Predecessores):** Um vetor que regista as referências de todos os outros nós que contêm escolhas a apontar para o nó atual.

Esta representação bidirecional permite à aplicação realizar mapeamento reverso instantâneo (*backward mapping*), fornecendo na barra lateral de propriedades os painéis “Vens de...” e “Vais para...”, facilitando ao autor a navegação e a validação rápida de interconexões sem ter de vasculhar visualmente o canvas.

A estrutura de dados JSON representativa de um nó e das suas ligações apresenta o seguinte formato:

```
1 {
2   "cenas": [
3     {
4       "id": "cena_praca",
5       "titulo": "A Praça Central",
6       "conteudo": "O sol brilha no mercado central.",
7       "saidas": [
8         { "texto_escolha": "Entrar na taberna", "destino": "cena_taberna" },
9         { "texto_escolha": "Aguardar um momento", "destino": "cena_praca" }
10      ],
11      "entradas": ["cena_praca", "cena_taberna"]
12    },
13    {
14      "id": "cena_taberna",
15      "titulo": "Interior da Taberna",
16      "conteudo": "O cheiro a vinho enche o ar.",
17      "saidas": [
18        { "texto_escolha": "Voltar lá fora", "destino": "cena_praca" }
19      ],
20      "entradas": ["cena_praca"]
21    }
22  ]
23 }
```

3.4 Parser Síncrono e Especificação Twee 3

O analisador desenvolvido em `tweeParser.js` (encontrado em `/src/utils/tweeParser.js`) realiza o mapeamento síncrono em tempo real entre a interface gráfica do React Flow e a representação textual no formato Twee 3 ([Interactive Fiction Technology Foundation, 2020](#)).

3.4.1 Importação e Conversão de Coordenadas em Zonas

No formato Twee 3 ([Interactive Fiction Technology Foundation, 2020](#)), as coordenadas de posição guardadas nos metadados JSON das passagens são sempre absolutas no espaço do ecrã:

$$(X_{\text{absoluto}}, Y_{\text{absoluto}})$$

Quando um nó é importado como filho de uma Zona de Agrupamento Visual (*ZoneNode*), o React Flow exige que a sua posição seja definida em coordenadas relativas ao canto superior esquerdo da zona mãe. O parser realiza esta conversão no momento da leitura:

$$X_{\text{relativo}} = X_{\text{absoluto}} - X_{\text{zona}}$$

$$Y_{\text{relativo}} = Y_{\text{absoluto}} - Y_{\text{zona}}$$

Ao exportar o grafo novamente para o formato `.twee`, o parser faz a conversão inversa para garantir a compatibilidade com outros compiladores de Twine tradicionais:

$$X_{\text{absoluto}} = X_{\text{zona}} + X_{\text{relativo}}$$

$$Y_{\text{absoluto}} = Y_{\text{zona}} + Y_{\text{relativo}}$$

3.4.2 Passagens Especiais

O parser reconhece passagens reservadas de metadados da especificação Twee 3:

- **:: StoryTitle:** Define o título do projeto.
- **:: StoryData:** Contém as configurações globais em formato JSON, incluindo o identificador único da história (`ifid`), o formato de compilação (`SugarCube`), o nó de arranque (`start`) e o nível de zoom.
- **:: StoryInit:** Nó especial não-narrativo onde são declaradas e inicializadas as variáveis de controlo de estado da simulação.

3.4.3 Modelo SugarCube (Ligações e Macros)

As arestas são extraídas do texto narrativo de cada nó através de expressões regulares que identificam ligações no formato clássico:

- **Arestas Simples:** `[[Cena2]]`

- **Arestas com Alias/Texto de Opção:** `[[Ir para a Taverna|cena_taberna]]` ou `[[Abrir porta->cena_interior]]`

As alterações de estado são definidas em macros como `<<set $moedas += 10>>`, e as condições lógicas que limitam a visibilidade de opções são declaradas através de blocos `<<if $tem_chave>> ... <</if>>`.

3.5 Interação Visual, Edição e Acessibilidade

3.5.1 Zonas de Agrupamento Visual

A implementação de Zonas como nós customizados de grupo (*ZoneNode*) exigiu uma resolução específica para um problema clássico de captura de cliques no React Flow. Como o contentor nativo das Zonas cobre uma área retangular considerável sobrepondo-se aos nós de cena interiores, os pointer events capturavam os cliques na tela, impedindo o arrasto ou seleção dos nós filhos.

A solução consistiu em definir a propriedade CSS `pointer-events: none` dinamicamente na configuração global do React Flow para o contentor da Zona em `App.jsx` (encontrado em `/src/App.jsx`). Em paralelo, no componente `ZoneNode.jsx` (encontrado em `/src/components/ZoneNode.jsx`), forçou-se `pointer-events: auto` exclusivamente nas pegadas de redimensionamento (*NodeResizer*) e na barra de cabeçalho da zona (utilizada para arrastar a zona em bloco com todos os seus filhos).

3.5.2 Arestas Curvadas com Waypoints

Para evitar a sobreposição caótica de ligações em grafos com elevado volume de cruzamentos, implementaram-se arestas customizadas baseadas em **Splines Catmull-Rom** (Catmull et al., 1974) em `EditableEdge.jsx` (encontrado em `/src/components/EditableEdge.jsx`). As arestas disponibilizam marcadores circulares (*waypoints*) interativos que o autor pode arrastar para curvar a linha no ecrã. A geometria da aresta é recalculada em tempo de execução gerando curvas Bézier cúbicas que contornam os nós do grafo.

3.5.3 Gestão de Variáveis Estilo Figma

O componente `VariablesModal.jsx` (encontrado em `/src/components/VariablesModal.jsx`) centraliza a gestão de variáveis globais e locais da história através de uma interface inspirada na ferramenta Figma. Quando uma variável é renomeada no painel global (ex: de `$ouro` para `$moedas`), um mecanismo automático executa uma expressão regular compilada:

```
1 new RegExp('\\$' + antigo + '\\b', 'g')
```

Esta regex varre e atualiza em tempo real todas as referências da variável dentro das macros `<<set>>`, `<<if>>` e inserções de texto em todos os nós da aplicação, prevenindo a quebra de referências em projetos grandes.

3.5.4 Acessibilidade para Autores Daltónicos

Tendo em conta que o autor principal da ferramenta, Miguel, apresenta daltonismo, desenhou-se uma estratégia de representação de fluxo que não depende da cor:

- **Arestas de Entrada:** Quando um nó é selecionado, as arestas que entram no nó mudam o seu padrão visual para uma linha tracejada de espessura elevada.
- **Arestas de Saída:** As ligações que saem do nó mantêm-se em traço contínuo com setas direcionais acentuadas a apontar para os destinos.
- **Alto Contraste:** Cores de contraste elevado e símbolos iconográficos acompanham o feedback visual em detrimento de códigos de cores simples (verde/vermelho).

3.6 Validação Estática e Simulação

3.6.1 Validador Estático de Lógica (BFS)

O ficheiro `storyValidator.js` (encontrado em `/src/utils/storyValidator.js`) executa testes estáticos sobre a estrutura do grafo através de uma travessia BFS (Cormen et al., 2009). A verificação analisa:

- **Nós Órfãos:** Nós inacessíveis a partir da cena de início da história (ignora nós rotulados com a etiqueta especial `secreto`).
- **Becos sem Saída (Dead Ends):** Distingue nós terminais legítimos (onde a história conclui propositadamente) de nós que contêm ligações que apontam para IDs de nós apagados ou inexistentes no grafo.
- **Traces de Arestas Bloqueadas:** Valida se existem condições lógicas impossíveis de satisfazer em determinados ramos.

3.6.2 Simulador em Tempo Real (PlayMode) e Smart Walker

O simulador interativo (`PlayMode.jsx` encontrado em `/src/components/PlayMode.jsx`) recria a experiência de jogo do leitor.

Durante a inicialização da simulação, os nós classificados como `css` têm o seu conteúdo concatenado e injetado na diretoria principal da página como um elemento de estilo `<style id="playmode-stylesheet">`, sendo removido ao encerrar a simulação. Os nós classificados como `javascript` são executados através de uma função isolada com a assinatura `new Function('state', 'State', 'setup', código)`, permitindo que scripts criados pelo autor manipulem diretamente a fita de dados da simulação.

Para testes automatizados de rotas, a inteligência do simulador implementa um algoritmo de **Exploração de Novidade (Smart Walker)** que substitui a seleção aleatória de escolhas por uma abordagem heurística:

1. O simulador mantém um conjunto de caminhos já visitados em memória (*Global Memory Set*).

2. Perante um nó com múltiplas escolhas, efetua-se uma poda da árvore de decisão (*Pruning*), dando prioridade absoluta às opções que conduzem a nós com menor taxa de visita histórica.
3. Permite simular caminhos de equilíbrio (como Cabo de Guerra ou mecânicas de sobrevivência) e identificar becos sem saída de forma automatizada e rápida.

4

Conclusão e Trabalho Futuro

Este capítulo encerra o relatório, apresentando um balanço das conquistas do projeto, as suas limitações e as principais perspectivas para desenvolvimentos futuros.

4.1 Conclusões

O desenvolvimento do **Plot in a Pot** alcançou com êxito os objetivos propostos. A plataforma conseguiu colmatar as falhas identificadas no estado da arte ao unir as vantagens de uma interface puramente visual baseada em grafos direcionados gerais à robustez lógica de variáveis e macros da especificação Twee 3 (SugarCube).

Ao implementar a representação em **Lista de Adjacência Bidirecional**, o editor conseguiu manter um desempenho fluido mesmo com o aumento do número de nós no canvas, facilitando a navegação reversa e o mapeamento de fluxos. O parser síncrono demonstrou ser fiável, mantendo a compatibilidade retroativa absoluta e resolvendo os problemas de cálculo de coordenadas espaciais para nós agrupados em zonas.

Do ponto de vista da acessibilidade, a criação de estilos visuais baseados em espessuras e tracejados de arestas, aliada à identidade neo-brutalista de alto contraste, provou ser uma solução eficaz para mitigar as limitações de utilização por pessoas daltónicas, assegurando que a distinção entre fluxos de entrada e saída é feita de forma geométrica e inequívoca.

Por fim, as ferramentas de validação baseadas em travessias BFS e o ambiente de simulação PlayMode (com injeção dinâmica de CSS e sandbox em Javascript) proporcionaram aos autores um ambiente de testes completo, robusto e livre de bloqueios inesperados.

4.2 Limitações da Implementação

Apesar do sucesso geral da solução, identificaram-se algumas limitações técnicas durante o ciclo de desenvolvimento:

1. **Análise Sintática baseada em Expressões Regulares:** O motor de análise lógica de macros e variáveis assenta fortemente no uso de expressões regulares (*Regex*). Embora seja rápido, este método apresenta fragilidades na leitura de expressões lógicas com múltiplos níveis de aninhamento de condicionais complexas ou parêntesis em SugarCube.
2. **Gestão de Pesos das Arestas:** O modelo atual suporta o conceito de grafos direcionados lógicos, mas ainda não permite a atribuição visual de pesos numéricos diretos nas arestas do grafo de forma a modelar custos ou probabilidades sem recorrer a macros textuais.
3. **Restrição de Destinos Estáticos de Hiperligações:** Ao contrário do Twine tradicional, que suporta o uso de variáveis como destinos dinâmicos de hiperligações (ex: `[[Seguir em frente|$caminho]]`), o **PiaP** não suporta esta funcionalidade. Como o editor precisa de analisar estaticamente o texto para extrair as arestas e renderizar visualmente o grafo no canvas, todos os caminhos e destinos de ligação têm de ser rigidamente declarados (*hardcoded*), representando um compromisso de flexibilidade em prol da clareza visual e da facilidade de auditar o grafo.

4.3 Trabalho Futuro

Para desenvolvimentos futuros no **Plot in a Pot**, propõem-se as seguintes linhas de investigação e implementação:

- **Migração para um Parser baseado em Árvore de Sintaxe Abstrata (AST):** Substituir a análise baseada em *Regex* por um analisador léxico e sintático (*tokenizer*) formal. Isto permitirá suportar o aninhamento ilimitado de blocos lógicos SugarCube e fornecer erros de sintaxe detalhados na linha exata de escrita.
- **Implementação Visual de Grafos Ponderados:** Introduzir um campo de edição direto nas arestas customizadas para definir pesos e custos de forma visual.
- **Exploração Automatizada com Agente Inteligente:** Expandir a lógica do Smart Walker do simulador para criar testes automáticos que percorram o grafo simulando escolhas em paralelo, gerando um relatório PDF de caminhos bem-sucedidos e falhados.
- **Integração com Modelos de Linguagem (LLM):** Refinar a funcionalidade de conversão e estruturação automática de histórias escritas em linguagem natural para o formato Twee, expandindo o suporte de formatos e melhorando a fiabilidade sintática e de posicionamento gerada por modelos locais.

Bibliography

Articy Software (2010). *articy:draft: Tool for storytelling and narrative design*. URL: <https://www.articy.com/>.

Bondy, J. A. e U. S. R. Murty (2008). *Graph Theory*. Vol. 244. Graduate Texts in Mathematics. Springer. DOI: [10.1007/978-1-84628-970-5](https://doi.org/10.1007/978-1-84628-970-5).

Catmull, Edwin e Raphael Rom (1974). «A class of local interpolating splines». Em: *Computer Aided Geometric Design*. Ed. por Robert E. Barnhill e Richard F. Riesenfeld. Academic Press, pp. 317–326.

Choice of Games (2026). *ChoiceScript License and Terms of Use*. URL: <https://www.choiceofgames.com/make-your-own-games/choicescript-intro/>.

Cormen, Thomas H. et al. (2009). *Introduction to Algorithms*. 3rd. MIT Press.

Inkle Studios (2016). *Ink: inkle’s open source scripting language for writing interactive narrative*. URL: <https://www.inklestudios.com/ink/>.

Interactive Fiction Technology Foundation (2020). *Twee 3 Specification*. URL: <https://github.com/iftechfoundation/twine-specs/blob/master/twee-3-specification.md>.

Interactive Fiction Technology Foundation (2026). *IFTF at GDC 2026: Recap and Reflections*. URL: <https://iftechfoundation.org/news/iftf-gdc-2026/>.

Kahn, Arthur B. (1962). «Topological sorting of large networks». Em: *Communications of the ACM* 5.11, pp. 558–562. DOI: [10.1145/368996.369025](https://doi.org/10.1145/368996.369025).

Klimas, Chris (2009). *Twine: An open-source tool for telling interactive, nonlinear stories*. URL: <https://twinery.org/>.

Meta Open Source (2013). *React: A JavaScript library for building user interfaces*. URL: <https://react.dev/>.

Montfort, Nick (2005). *Twisty Little Passages: An Approach to Interactive Fiction*. Cambridge, MA: MIT Press.

NarrativeFlow (2023). *NarrativeFlow: Visual scripting for interactive stories*. URL: <https://narrativeflow.dev/>.

Secret Lab (2015). *Yarn Spinner: The friendly tool for game dialogue*. URL: <https://yarnspinner.dev/>.

Statista Research Department (2025). *Most used web frameworks among developers worldwide*. URL: <https://www.statista.com/>.

Tailwind Labs (2017). *Tailwind CSS: A utility-first CSS framework for rapid UI development*. URL: <https://tailwindcss.com/>.

TheToolpicker Team (2026). *Best Interactive Fiction Tools*. URL: <https://thetoolpicker.com/>.

webkid.io (2021). *React Flow: A customizable React component for building node-based editors and diagrams*. URL: <https://reactflow.dev/>.

Apêndices



Showcasing the First Appendix

i Orientação Para a Escrita

Appendices contain supplementary material **created by the author** that enhances the reader's understanding of the dissertation while not being essential for following the primary narrative. These sections often include detailed tables, figures, complex calculations, or materials like survey questions and interview transcripts produced in the course of the research. The appendices allow readers to explore the research in greater detail, offering a deeper insight into methods and findings without interrupting the main body of work.

B

Showcasing the Second Appendix

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Anexos



Showcasing the First Annex

i Orientação Para a Escrita

Annexes are supplementary sections in a dissertation that provide additional information or external documents not essential to the main arguments but that support or complement the research. Unlike appendices, **annexes generally contain material that was not developed by the author**, such as reports, legal documents, or published datasets from external sources. This information is placed separately to keep the main content concise, allowing readers access to relevant external references without disrupting the dissertation's flow.

